Requisitos

- `Python`
- `Biopython 1.85`
- `Numpy`

Ejercicios prácticos Tema 2A: Sesión 1

Con estos ejercicios, nos familiarizaremos con los algoritmos estudiados en el tema, para poder abordar la segunda iteración del pipeline entregable de la asignatura.

En este caso, al tratarse de ejercicios introductorios no entregables, trabajaremos en un Jupyter Notebook.

Ejercicio 1: Creación de un Dot Plot Simple

Implementa una función en Python que tome dos secuencias como entrada y genere una matriz de coincidencias (dot plot) usando `matplotlib`.

Pasos sugeridos:

1. Recibir dos secuencias como argumentos de la función.
2. Crear una matriz numpy de dimensiones correspondientes a la longitud de las secuencias.
3. Rellenar la matriz con unos y ceros, donde 1 representa una coincidencia y 0 una diferencia.
4. Representar la matriz gráficamente con la función `imshow` de `matplotlib`.

Ejercicio 2: Implementación simplificada de Needleman-Wunsch

Implementa una versión básica del algoritmo de Needleman-Wunsch para alinear dos secuencias de ADN con penalización de gaps.

Objetivos:

- Construir la matriz de puntuación.
- Rellenarla siguiendo la recurrencia del algoritmo.
- Devolver la matriz completa y la puntuación del alineamiento.

Pasos sugeridos:

1. Que la función reciba dos secuencias y valores para match, mismatch y gap.
2. Crear la matriz de puntuación e inicializarla correctamente.
3. Rellenar la matriz siguiendo el criterio del algoritmo.
4. Mostrar la matriz final (sin backtracking aún).

Entrada:

```
def needleman_wunsch(seq1, seq2, match, mismatch, gap):
```

Donde:

- `seq1`: cadena de caracteres
- `seq2`: cadena de caracteres
- `match`: puntuación cuando los caracteres coinciden (ej. `5`)
- `mismatch`: puntuación cuando no coinciden (ej. `-3`)
- `gap`: penalización por introducir un hueco (ej. `-4`)

Salida

La función debe devolver:

- Una matriz de puntuación de tamaño $(\texttt{len(seq1)} + 1) \times (\texttt{len(seq2)} + 1)$ (puede representarse como una lista de listas de Python, o un array de `NumPy`).
- La puntuación final del alineamiento (valor en la esquina inferior derecha)

Recuerda el algoritmo

Para cada posición $N(i, j)$:

$$N(i,j) = \max\{\, N\ (i-1, j-1) + s(x_i, y_j), N(i-1, j) + gap, N(i, j-1) + gap$$

Donde:

- $s(x_i, y_j) = match$ si $x_i = y_j$
- $s(x_i, y_j) = mismatch$ si $x_i \neq y_j$

Ejemplo

Entrada:

```
seq1 = "MKTAYIAKQRQISFVKSHFSRQLEER"
seq2 = "MKALYIAKKRQTSFVKSHFSNQLEQ"
match = 5
mismatch = -2
gap = -3
```

Salida esperada:

	-	M	K	A	L	Y	I	A	K	K	R	Q	T	S	F	V	K	S	H	F	S	N	Q	L	E	Q
-	0	-3	-6	-9	-12	-15	-18	-21	-24	-27	-30	-33	-36	-39	-42	-45	-48	-51	-54	-57	-60	-63	-66	-69	-72	-75
M	-3	5	2	-1	-4	-7	-10	-13	-16	-19	-22	-25	-28	-31	-34	-37	-40	-43	-46	-49	-52	-55	-58	-61	-64	-67
K	-6	2	10	7	4	1	-2	-5	-8	-11	-14	-17	-20	-23	-26	-29	-32	-35	-38	-41	-44	-47	-50	-53	-56	-59
T	-9	-1	7	8	5	2	-1	-4	-7	-10	-13	-16	-12	-15	-18	-21	-24	-27	-30	-33	-36	-39	-42	-45	-48	-51
A	-12	-4	4	12	9	6	3	4	1	-2	-5	-8	-11	-14	-17	-20	-23	-26	-29	-32	-35	-38	-41	-44	-47	-50
Y	-15	-7	1	9	10	14	11	8	5	2	-1	-4	-7	-10	-13	-16	-19	-22	-25	-28	-31	-34	-37	-40	-43	-46
I	-18	-10	-2	6	7	11	19	16	13	10	7	4	1	-2	-5	-8	-11	-14	-17	-20	-23	-26	-29	-32	-35	-38
A	-21	-13	-5	3	4	8	16	24	21	18	15	12	9	6	3	0	-3	-6	-9	-12	-15	-18	-21	-24	-27	-30
K	-24	-16	-8	0	1	5	13	21	29	26	23	20	17	14	11	8	5	2	-1	-4	-7	-10	-13	-16	-19	-22
Q	-27	-19	-11	-3	-2	2	10	18	26	27	24	28	25	22	19	16	13	10	7	4	1	-2	-5	-8	-11	-14
R	-30	-22	-14	-6	-5	-1	7	15	23	24	32	29	26	23	20	17	14	11	8	5	2	-1	-4	-7	-10	-13
Q	-33	-25	-17	-9	-8	-4	4	12	20	21	29	37	34	31	28	25	22	19	16	13	10	7	4	1	-2	-5
I	-36	-28	-20	-12	-11	-7	1	9	17	18	26	34	35	32	29	26	23	20	17	14	11	8	5	2	-1	-4
S	-39	-31	-23	-15	-14	-10	-2	6	14	15	23	31	32	40	37	34	31	28	25	22	19	16	13	10	7	4
F	-42	-34	-26	-18	-17	-13	-5	3	11	12	20	28	29	37	45	42	39	36	33	30	27	24	21	18	15	12
V	-45	-37	-29	-21	-20	-16	-8	0	8	9	17	25	26	34	42	50	47	44	41	38	35	32	29	26	23	20
K	-48	-40	-32	-24	-23	-19	-11	-3	5	13	14	22	23	31	39	47	55	52	49	46	43	40	37	34	31	28
S	-51	-43	-35	-27	-26	-22	-14	-6	2	10	11	19	20	28	36	44	52	60	57	54	51	48	45	42	39	36
H	-54	-46	-38	-30	-29	-25	-17	-9	-1	7	8	16	17	25	33	41	49	57	65	62	59	56	53	50	47	44
F	-57	-49	-41	-33	-32	-28	-20	-12	-4	4	5	13	14	22	30	38	46	54	62	70	67	64	61	58	55	52
S	-60	-52	-44	-36	-35	-31	-23	-15	-7	1	2	10	11	19	27	35	43	51	59	67	75	72	69	66	63	60
R	-63	-55	-47	-39	-38	-34	-26	-18	-10	-2	6	7	8	16	24	32	40	48	56	64	72	73	70	67	64	61
Q	-66	-58	-50	-42	-41	-37	-29	-21	-13	-5	3	11	8	13	21	29	37	45	53	61	69	70	78	75	72	69
L	-69	-61	-53	-45	-37	-40	-32	-24	-16	-8	0	8	9	10	18	26	34	42	50	58	66	67	75	83	80	77
E	-72	-64	-56	-48	-40	-39	-35	-27	-19	-11	-3	5	6	7	15	23	31	39	47	55	63	64	72	80	88	85
E	-75	-67	-59	-51	-43	-42	-38	-30	-22	-14	-6	2	3	4	12	20	28	36	44	52	60	61	69	77	85	86
R	-78	-70	-62	-54	-46	-45	-41	-33	-25	-17	-9	-1	0	1	9	17	25	33	41	49	57	58	66	74	82	83

```
Score: 83
```

Ejercicios prácticos Tema 2A: Sesión 2

Ejercicio 3: backtracking

Añade a la implementación del ejercicio 3 la creación y recorrido de la matriz de backtracking para obtener el alineamiento final.

Pasos sugeridos:

1. Establece un código numérico que incluir en la matriz T (no podemos usar flechas).
2. Establece los criterios de inicio y de fin para el recorrido de la matriz (backtracking).

Ejercicio 4: Implementación Manual de Smith-Waterman

Modifica el ejercicio anterior para implementar el algoritmo de Smith-Waterman (alineamiento local).

Pasos sugeridos:

1. Modificar la fórmula de cálculo para que los valores negativos sean reemplazados por 0.
2. Modificar el backtracking para empezar por la mayor puntuación y terminar en 0.